

What is a language?

- **Alphabet** – finite character set (Σ)
- **String** – finite sequence of characters – can be ε , the empty string (Some texts use λ as the empty string)
- **Language** – possibly infinite set of strings over some alphabet – can be $\{ \}$, the empty language.

Suppose $\Sigma = \{a,b,c\}$. Some languages over Σ could be:

- $\{aa,ab,ac,bb,bc,cc\}$
- $\{ab,abc,abcc,abccc,\dots\}$
- $\{\varepsilon\}$
- $\{\}$
- $\{a,b,c,\varepsilon\}$
- $\dots$

Why do we care about Regular Languages?

- Formally describe tokens in the language
 - Regular Expressions
 - NFA
 - DFA
- Regular Expressions → finite automata
- Tools assist in the process

Regular Expressions

The **regular expressions** over finite Σ are the strings over the alphabet $\Sigma + \{ \emptyset, (, |, * \}$ such that:

1. $\emptyset$ (empty set) is a regular expression for the empty set
2. ε is a regular expression denoting $\{ \varepsilon \}$
3. a is a regular expression denoting set $\{ a \}$ for any a in Σ

Regular Expressions

4. If P and Q are regular expressions over Σ , then so are:

- **$P \mid Q$ (union)**

If P denotes the set $\{a, \dots, e\}$, Q denotes the set $\{0, \dots, 9\}$ then
 $P \mid Q$ denotes the set $\{a, \dots, e, 0, \dots, 9\}$

- **PQ (concatenation)**

If P denotes the set $\{a, \dots, e\}$, Q denotes the set $\{0, \dots, 9\}$ then
 PQ denotes the set $\{a0, \dots, e0, a1, \dots, e9\}$

- **Q^* (closure)**

If Q denotes the set $\{0, \dots, 9\}$ then Q^* denotes the set
 $\{\epsilon, 0, \dots, 9, 00, \dots, 99, \dots\}$

Examples

If $\Sigma = \{a, b\}$

- $(a \mid b)(a \mid b)$
- $(a \mid b)^*b$
- $a^*b^*a^*$
- a^*a (also known as a^+)
- $(ab^*) \mid (a^*b)$

- $\Sigma = \{\mathbf{a}, \mathbf{b}\}$

This means the language uses only the symbols **a** and **b**.

1. **$(a \mid b)(a \mid b)$**

- This means: **a or b**, followed by **a or b**.
- Matches: **aa, ab, ba, bb**
- This describes all **strings of length 2** over $\{a, b\}$.

2. $(a \mid b)^*b$

- This means: **any string** of a's and b's ending in a **b**.
- $(a \mid b)^*$ matches any string (including empty),
- and **b** enforces that it must end with a **b**.
- Matches: **b, ab, aab, bab, abba, etc.**
- **Does not match: a, aa, aba**

3. **$a^*b^*a^*$**

- Zero or more **a's**, followed by zero or more **b's**, followed by zero or more **a's**.
- Matches: **b**, **aaabaaa**, **aabbaaa**, **aab**, **bbaaa**, **aa**, etc.
- This allows **a's** on both sides of the **b block**.

4. a^*a (also known as a^+)

- One or more a 's.
- a^* = zero or more a 's; adding a final a ensures at least one occurs.
- This is the same as the shorthand a^+ = one or more a 's.
- Matches: a , aa , aaa , etc.
- **Does not match:** empty string

5. $(ab^*)|(a^*b)$

- This is an **OR** of two expressions:
 - ab^* : an **a** followed by **zero or more** b's $\rightarrow a, ab, abb, abbb, \text{etc.}$
 - a^*b : **zero or more a's**, followed by **b** $\rightarrow b, ab, aab, \text{etc.}$
- Matches both kinds: $ab, a, abb, b, aab, \text{etc.}$
- Union includes both patterns.

Nondeterministic Finite Automata

A **nondeterministic finite automaton** (NFA) is a mathematical model that consists of

1. A set of states S
2. A set of input symbols Σ
3. May have multiple transition function that maps state/symbol pairs to a set of states:

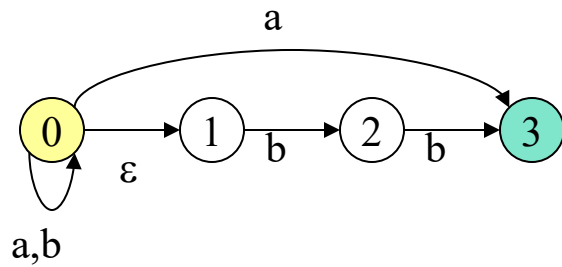
$$S \times \{\Sigma + \epsilon\} \rightarrow \text{set of } S$$

4. A special state s_0 called the start state
5. Have epsilon **ϵ -transitions or lambda transitions**
6. A set of states F (a subset of S) of final states

INPUT: string

OUTPUT: yes or no

Example NFA



$S = \{0,1,2,3\}$

$S_0 = 0$

$\Sigma = \{a,b\}$

$F = \{3\}$

Transition Table:

STATE	a	b	ϵ
0	0,3	0	1
1		2	
2		3	
3			

Key Definitions:

- **States (S):** $\{0, 1, 2, 3\}$
- **Start state (S_0):** 0
- **Alphabet (Σ):** $\{a, b\}$
- **Final/Accepting state (F):** $\{3\}$

•**State 0:**

- On '**a**', go to **state 0** (loop) and **state 3** (non-deterministic jump).
- On '**b**', go to **state 0** (loop).
- On ϵ (**epsilon**), go to **state 1** (without consuming input).

•**State 1:**

- On '**b**', go to **state 2**.

•**State 2:**

- On '**b**', go to **state 3**.

•**State 3:**

- Is a **final state**. No transitions from here.

Graph Explanation

- The **yellow state 0** is the **start state**.
- It loops on **a** and **b**.
- It can transition via ϵ to **state 1**, starting a path through $b \rightarrow b$ to reach the **accepting state 3** (turquoise).
- **State 0** also non-deterministically transitions directly to **state 3** on input **a**, giving alternate acceptance paths.

What Strings Are Accepted?

Let's trace a few:

1. "a"

1. From 0, on "a", go to 3 → **Accepted**

2. "bb"

1. From 0, ϵ to 1 → b → 2 → b → 3 → **Accepted**

3. "ab"

1. From 0, on "a" → go to 0 and 3
2. From 3: finished (1 character) → **Accepted**
3. From 0 → on "b", stay in 0 → not accepted yet
4. But because 3 is accepting → **Accepted**

4. "aaa"

1. From 0 → loops on "a" each time, stays in 0
2. Never reaches 3 via ϵ unless a switch is made
3. But on any "a", it could nondeterministically jump to 3 → **Accepted**

- This NFA is **nondeterministic**, allowing **multiple paths**.
- It accepts strings like "a", "bb", "ab", "bba", etc.
- **ϵ -transitions** allow jumping to another computation path without consuming a character.
- The final state is **3**, and any path that leads to it **accepts the string**.

Deterministic Finite Automata

A **deterministic finite automaton** (DFA) is a mathematical model that consists of

1. A set of states S
2. A set of input symbols Σ
3. A Unique transition function that maps state/symbol pairs to a state:

$$S \times \Sigma \rightarrow S$$

4. A special state s_0 called the start state
5. A set of states F (a subset of S) of final states

INPUT: string

OUTPUT: yes or no

Deterministic Finite Automata (DFAs)

Formal Definition, Structure,
Example, and Transition
Representation

Definition

- A Deterministic Finite Automaton (DFA) is a finite automaton with:
- No ϵ (epsilon) transitions
- Exactly one transition per state-symbol pair
- It functions as a finite state machine that precisely recognizes a regular expression (RE).

2. Components of a DFA

- Formally, a DFA is a 5-tuple:
- $D = (Q, \Sigma, \delta, q_0, F)$

Where:

- Q : Finite set of states
- Σ : Input alphabet
- δ : Transition function $Q \times \Sigma \rightarrow Q$
- q_0 : Start state
- F : Set of accepting (final) states

How a DFA Works

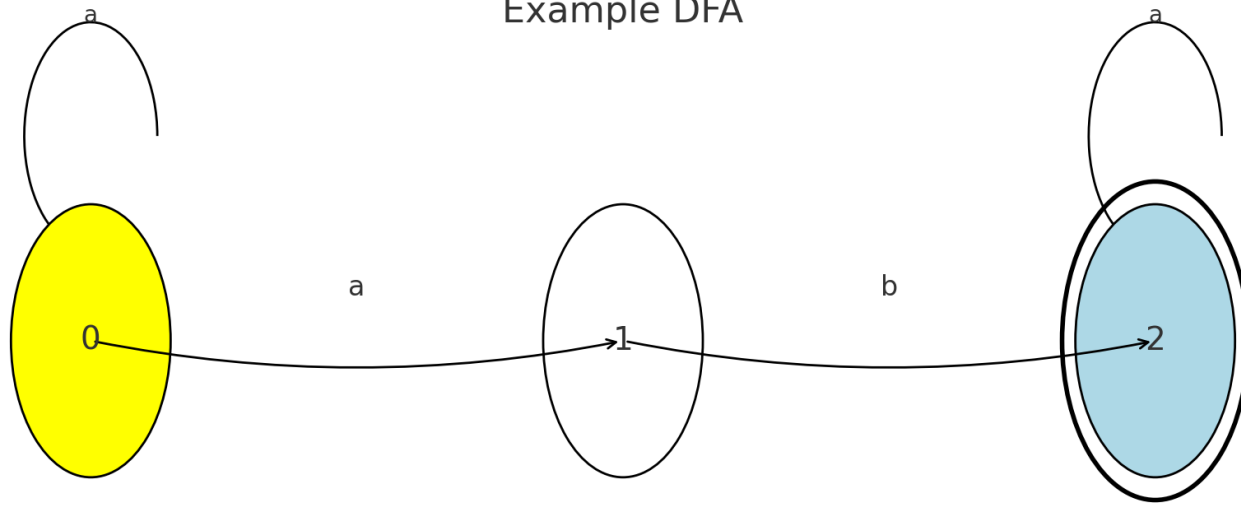
- A DFA accepts a string if:
- Starting at q_0 , it reads the string symbol by symbol
- For each input, it follows a defined transition
- If the final state reached is in F , the string is accepted
- Otherwise, it is rejected

Transition Table Representation

- A DFA can also be expressed using a transition table T :
- $T[s, c]$ gives the next state when in state s with input c
- This format helps in systematically defining all transitions

DFA Diagram

Example DFA



DFA Recognizing Strings with 'ab'

Includes optional leading and trailing 'a's

DFA Structure Explanation

States:

- q_0 : Start state
- q_1 : After reading 'a'
- q_2 : Final state after 'ab'

Transitions:

- $q_0 \rightarrow q_1$ on 'a'
- $q_1 \rightarrow q_2$ on 'b'
- $q_2 \rightarrow q_2$ on 'a'
- $q_0 \rightarrow q_0$ on 'a'
- q_2 is the accepting state (double circle)

How This DFA Works

- Starts at q_0
- Requires at least one occurrence of 'ab' to reach q_2
- Once in q_2 , it stays there on any number of 'a's
- Extra 'a's before 'ab' are handled by looping at q_0

Accepted and Rejected Strings

Accepted Strings:

- ab
- aab
- aaba
- aaabaaa

Rejected Strings:

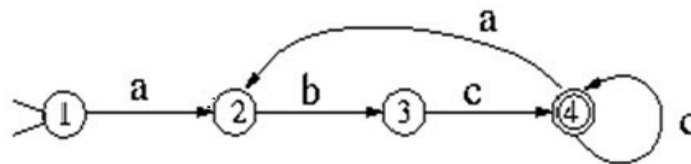
- a
- aa
- aaa
- b

DFA Explanation: Recognizing $(abc)^+c^*$

Deterministic Finite Automaton
(DFA) Analysis

DFA Diagram

- The DFA below accepts strings of the form $(abc)^+$ followed by optional c's.



DFA Transitions

- Transitions:
- - $q1 \xrightarrow{a} q2$
- - $q2 \xrightarrow{b} q3$
- - $q3 \xrightarrow{c} q4$ (accepting)
- - $q4 \xrightarrow{c} q4$ (loop)
- - $q4 \xrightarrow{a} q2$ (restart abc cycle)

Accepted Strings

- Examples of Accepted Strings:
- ✓ abc
- ✓ abcc
- ✓ abcabc
- ✓ abcabcabc
- ✓ abcabccccc

Rejected Strings

- Examples of Rejected Strings:
- **X** a
- **X** ab
- **X** ac
- **X** abcab
- **X** abcabca

Language Description and Regular Expression

- The DFA accepts strings that:
 - - Start with one or more occurrences of 'abc'
 - - May be followed by any number of 'c's
- Regular Expression:
- $(abc)^+c^*$